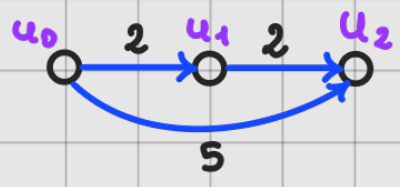
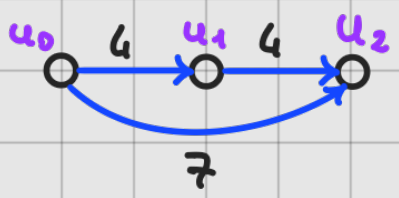
Tutorato AsD ~ L11 — 28/05/2026

Esercizio 1. Sia $G = (V, E)$ un grafo diretto con funzione peso $w\colon E \to \mathbb{R}$ e sia $a \in \mathbb{R}$. Considerate una nuova funzione peso data da $w'(u,v) = w(u,v) + a$ per ogni $(u,v) \in E$. I cammini minimi in G considerando prima la funzione peso w e poi quella w' sono gli stessi? Cosa succede se $w'(u,v) = a \cdot w(u,v)$ per ogni arco $(u,v) \in E$ e un certo $a > 0$?

I G, w $\quad$ G, w' $\quad w'(u,v) = w(u,v) + 2$





$\delta(u_0, u_2) = 4$ $\quad$ $\delta(u_0, u_2) = 7$

cammino minimo $\quad$ p non è più cammino minimo

$p = \langle u_0, u_1, u_2 \rangle$ $\quad$ $p' = \langle u_0, u_2 \rangle$

In questo caso i cammini minimi possono cambiare

II $w'(u,v) = a \cdot w(u,v)$

Sia p cammino in G

(es. conversione miglia/Km)

↳ Rescaling di tutti i cammini della stessa costante a

↳ Cammini minimi rispetto a w restano minimi rispetto a w'

Esercizio 2. Fornite un algoritmo che, dato un grafo diretto aciclico e pesato, trovi un cammino di peso *massimo* da una sorgente s ad un vertice raggiungibile da s. Il tempo di esecuzione deve essere $O(n+m)$.

```
DAG-SHORTEST-PATHS(G, w, s)
1  topologically sort the vertices of G
2  INITIALIZE-SINGLE-SOURCE(G, s)
3  for each vertex u ∈ G.V, taken in topologically sorted order
4      for each vertex v in G.Adj[u]
5          RELAX(u, v, w)

TOPOLOGICAL-SORT(G)
1  call DFS(G) to compute finish times v.f for each vertex v
2  as each vertex is finished, insert it onto the front of a linked list
3  return the linked list of vertices

INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0

RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2      v.d = u.d + w(u, v)
3      v.π = u
```

$G=(V,E)$ diretto aciclico con f. peso w; s, u fissi.

Voglio cammino di peso massimo da s a u.

Sia $w': E \to \mathbb{R}$ con $w'(u,v) = -w(u,v)$. Questo trucco funziona solo su DAG

Considero G con f. peso w' e lancio

DAG-SHORTEST-PATHS(G, w', s) $\to O(n+m)$

Ottengo cammini minimi (rispetto a w') da s a tutti i vertici raggiungibili da s totale $O(n+m)$

↳ Corrispondono a cammini massimi (rispetto a w) da s a tutti i vertici raggiungibili da s.

Esercizio 3. Modificate l'algoritmo di Bellman-Ford in modo che $v.d$ venga posto a $-\infty$ per tutti i vertici v per i quali esiste un ciclo di peso negativo in qualche cammino dalla sorgente a v.

Dopo le n-1 iterazioni di Bellman-Ford, usiamo la condizione di rilassamento per individuare i vertici "contaminati" da cicli negativi, poi propaghiamo $-\infty$ con BFS.

```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i = 1 to |G.V| − 1
3      for each edge (u, v) ∈ G.E
4          RELAX(u, v, w)
5  for each edge (u, v) ∈ G.E
6      if v.d > u.d + w(u, v)
7          return FALSE
8  return TRUE
```

→

```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i = 1 to |G.V| − 1
3      for each edge (u, v) ∈ G.E
4          RELAX(u, v, w)
   L ← ∅
5  for each edge (u, v) ∈ G.E
*6     if v.d > u.d + w(u, v)
           L ← L ∪ {v}
   For each v ∈ L
       For each u reachable from v via BFS(G,v)
           u.d ← -∞
```

CORRETTEZZA

P_1: Dopo le righe 1-4 di B-F, $v.d > u.d + w(u,v)$ per qualche $(u,v) \in E$

P_2: $\exists$ un ciclo di peso negativo in qualche cammino da s a v.

$$P_1 <=> P_2$$

$P_1 => P_2$: Se $\exists$ un ciclo negativo in un cammino da s a v, allora dopo le (n-1) iterazioni $\exists$ un $(u,v) \in E$ che viola la condizione di rilassamento

$P_1 <= P_2$: Se dopo le (n-1) iterazioni $\exists$ un $(u,v) \in E$ che viola la condizione di rilassamento, allora $\exists$ un ciclo negativo in qualche cammino da s a v.

(=>) Se ∃ un ciclo negativo c in un cammino da s a v.

Sia $c = \langle v_0, v_1, \dots, v_k \rangle$ con $v_0 = v_k$ e $w(c) < 0$.

Dal 2° THM di correttezza di B-F, dopo le righe 1-4

$\exists i$ t.c. $v_i.d > v_{i-1}.d + w(v_{i-1}, v_i)$

↳ Il check alla riga * restituisce vero e qualche vertice di c finisce in L. Poiché v é raggiungibile da c, BFS imposta $v.d = -\infty$.

(<=) Se dopo le righe 1-4 si ha $v.d > u.d + w(u,v)$ per qualche $(u,v) \in E$.

Allora $u.d \neq +\infty$ (altrimenti la disuguaglianza non varrebbe), $\times v.d > +\infty \times$

quindi u é raggiungibile da s. Distinguiamo due casi sul cammino da s a u:

(A) ∃ un ciclo negativo nel cammino $s \rightsquigarrow u$. Allora posso prendere quel cammino e attaccarci in coda l'arco (u,v), ottenendo cosí un cammino $s \rightsquigarrow u \to v$ che contiene un ciclo negativo. La Tesi é dimostrata.

(B) nessun ciclo negativo nel cammino $s \rightsquigarrow u$. Allora per il 1° THM di correttezza di B-F si ha $u.d = \delta(s,u)$. Supponiamo per assurdo che anche nessun cammino $s \rightsquigarrow v$ contenga cicli negativi: allora analogamente $v.d = \delta(s,v)$. Ma per def. di distanza minima vale sempre $\delta(s,v) \leq \delta(s,u) + w(u,v)$, cioè $v.d \leq u.d + w(u,v)$ → CONTRADDIZIONE CON L'IP.
Quindi ∃ un ciclo negativo in qualche cammino da s a v.

Esercizio 4. Supponete di modificare la riga 4 dell'algoritmo di Dijkstra con

while $|Q| > 1$.

Il ciclo sarà quindi eseguito $n - 1$ volte, anziché n volte. Il nuovo algoritmo è corretto?

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = ∅
4  for each vertex u ∈ G.V
5      INSERT(Q, u)
6  while Q ≠ ∅ → |Q|>1
7      u = EXTRACT-MIN(Q)
8      S = S ∪ {u}
9      for each vertex v in G.Adj[u]
10         RELAX(u, v, w)
11         if the call of RELAX decreased v.d
12             DECREASE-KEY(Q, v, v.d)
```

L'algoritmo rimane corretto. | Non c'è guadagno reale in termini di tempo di esecuzione.

Sia v l'unico vertice che NON viene estratto da Q.
Distinguiamo 2 casi:

I) v NON è raggiungibile da s.

Dalla procedura INITIALIZE-SINGLE-SOURCE si ha $v.d = +\infty = \delta(s,v)$.

II) v è raggiungibile da s.

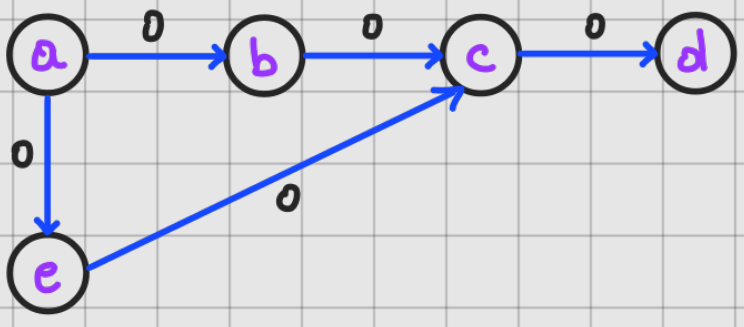
Sia $p = \langle s, \dots, u, v \rangle$ un cammino minimo da s a v con ultimo arco (u,v) (potrei avere $u = s$).

Quando u viene estratto da Q, per l'invariante di Dijkstra si ha $u.d = \delta(s,u)$. A quel punto viene rilassato l'arco (u,v), ottenendo $v.d = \delta(s,v)$ per la proprietà della convergenza.

In entrambi i casi, al termine, $v.d = \delta(s,v)$.

Esercizio 5. Vogliamo verificare come non sia possibile usare la Proprietà del rilassamento del cammino per dimostrare direttamente la correttezza dell'algoritmo di Dijkstra (ricordate invece come questa fosse la strategia usata per la correttezza di Bellman-Ford). Fornite quindi un esempio di grafo diretto per il quale Dijkstra non rilassa gli archi di un cammino minimo nell'ordine.

Consideriamo il grafo seguente con tutti i pesi uguali a 0:



Tutti i pesi sono 0. Un possibile ordine di estrazione da Q è: a, e, c, b, d

↳ Si estrae a: b.d = 0 e e.d = 0

↳ Si estrae e (parità con b rotta arbitrariamente): c.d = 0

↳ Si estrae c: l'arco (c,d) viene rilassato → d.d = 0

↳ Si estrae b: l'arco (b,c) viene rilassato (dopo (c,d)!)

↳ Si estrae d.

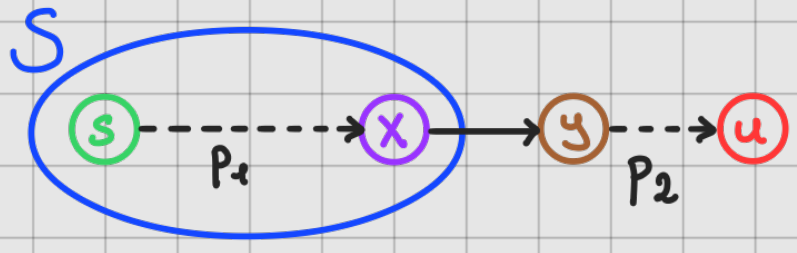
Il cammino minimo $\langle a,b,c,d \rangle$ ha i propri archi rilassati nell'ordine (a,b), (c,d), (b,c): l'arco (c,d) è rilassato prima di (b,c), violando la proprietà del rilassamento del cammino.

Esercizio 6 (Piccola generalizzazione di Dijkstra). Supponete di avere un grafo diretto pesato $G = (V, E)$ in cui gli archi che escono dalla sorgente s possono avere pesi negativi, i pesi di tutti gli altri archi sono non-negativi e non ci sono cicli di peso negativo. Dimostrate che, per un tale G, l'algoritmo di Dijkstra trova correttamente i cammini minimi da s.

La dimostrazione di correttezza di Dijkstra è identica a quella standard, tranne un unico punto: la disuguaglianza $\delta(s,y) \leq \delta(s,u)$.

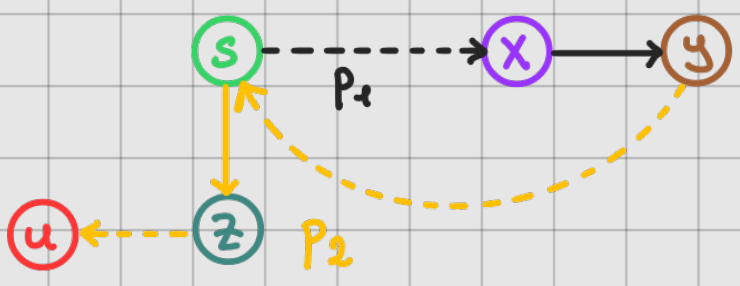
Ricordiamo il contesto: indichiamo con S l'insieme dei vertici estratti da Q.

Quando si sta per estrarre $u \notin S$, sia $p = p_1 \rightsquigarrow x \to y \rightsquigarrow p_2$ il cammino minimo da s a u, dove $x \in S$ e $y \notin S$ è il primo vertice fuori da S lungo p. ((x,y))



Dobbiamo verificare che $\delta(s,y) \leq \delta(s,u)$, cioè che tutti gli archi del sottocammino $p_2 = y \rightsquigarrow u$ abbiano peso ≥ 0.

Nel nuovo scenario, gli unici archi che possono avere peso negativo sono quelli uscenti da s. Supponiamo per assurdo che un arco di p_2 abbia peso negativo: deve allora uscire da s, supponiamo $(s,z) < 0$.



Otteniamo un ciclo, ma p NON contiene alcun ciclo.

Esercizio 7. Create un algoritmo con tempo di esecuzione $O(n + m)$ per contare il numero totale di cammini in un grafo diretto aciclico.

Per ogni vertice u, definiamo $u.paths$ come il numero di cammini in G che hanno u come vertice iniziale.

La ricorrenza è:

$$u.paths = 1 + \sum_{v \in G.Adj[u]} v.paths$$

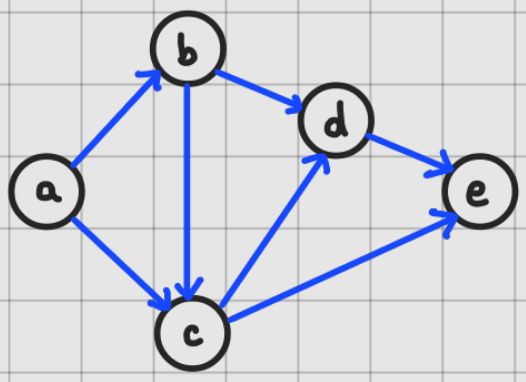
Il termine 1 conta il cammino che inizia e termina in u (cammino di lunghezza 0). Il termine $v.paths$ conta i cammini che escono da u tramite (u,v) e proseguono a partire da v.

Calcoliamo la ricorrenza in ordine topologico inverso, così quando elaboriamo u tutti i $v \in G.Adj[u]$ hanno già il valore corretto.

```
COUNT-PATHS(G)
    for each u ∈ G.V                                   O(n)
        u.paths = 0
    TOPOLOGICAL-SORT(G)                                O(n+m)
    for each u ∈ G.V in ordine topologico inverso      O(m)
        for each v ∈ G.Adj[u]
            u.paths += v.paths
        u.paths += 1
    num = 0
    for each u ∈ G.V                                   O(n)
        num += u.paths
```

totale: $O(n+m)$

ESEMPIO



p len 0: V → 5 (|V|)

p len 1: E → 7 (|E|)

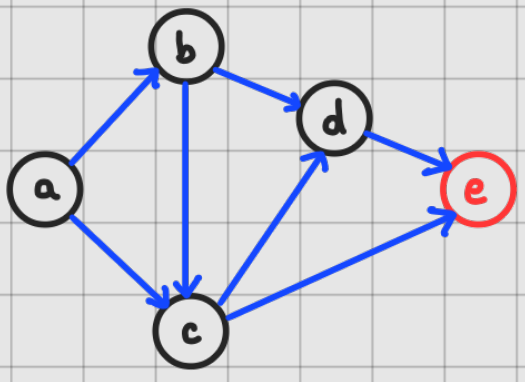
p len 2: {<a,b,c>, <a,b,d>, <a,c,d>, <a,c,e>, <b,c,d>, <b,c,e>, <b,d,e>, <c,d,e>} → 8

p len 3: {<a,b,c,d>, <a,b,c,e>, <a,c,d,e>, <b,c,d,e>} → 5

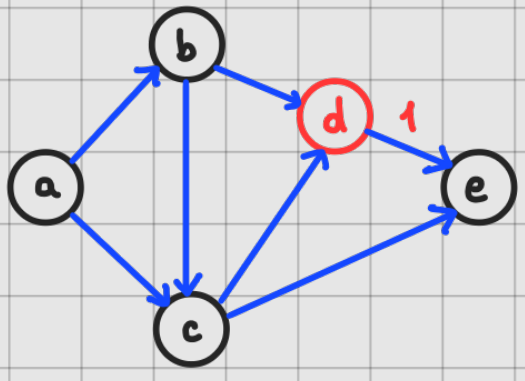
p len 4: {<a,b,c,d,e>} → 1

totale: 5+7+8+5+1 = 26

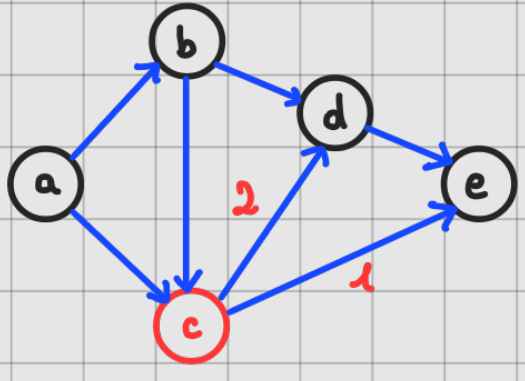
ORDINE: e, d, c, b, a



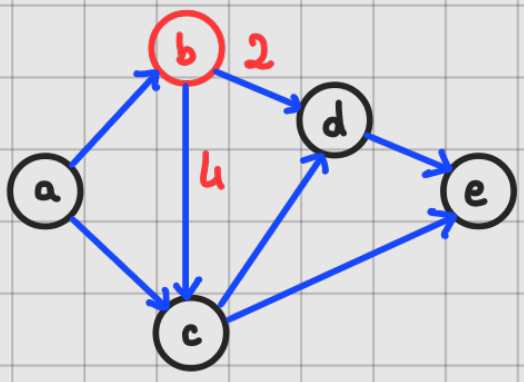
	paths
a	
b	
c	
d	
e	1



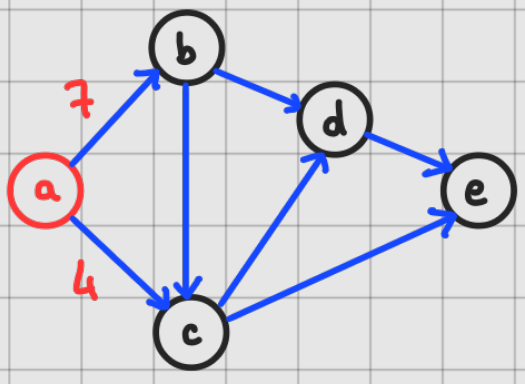
	paths
a	
b	
c	
d	2
e	1



	paths
a	
b	
c	4
d	2
e	1



	paths
a	
b	7
c	4
d	2
e	1



	paths
a	12
b	7
c	4
d	2
e	1

	paths
a	12
b	7
c	4
d	2
e	1
	26 ✓

Esercizio 8. Sia (u, v) un arco di peso minimo in un grafo connesso G. Dimostrate che (u, v) appartiene a qualche albero di connessione minimo.

Lanciamo Kruskal su G e, se ∃ più archi con lo stesso peso minimo di (u,v), poniamo (u,v) in testa all'ordinamento degli archi.

Poiché w(u,v) è il peso minimo, (u,v) è il primo arco esaminato da Kruskal. Quando Kruskal lo esamina, u e v si trovano in componenti distinte della foresta (all'inizio ogni nodo è una componente isolata), quindi (u,v) viene aggiunto ad A.

Per la correttezza di Kruskal, A al termine dell'algoritmo è un MST di G e $(u,v) \in A$.

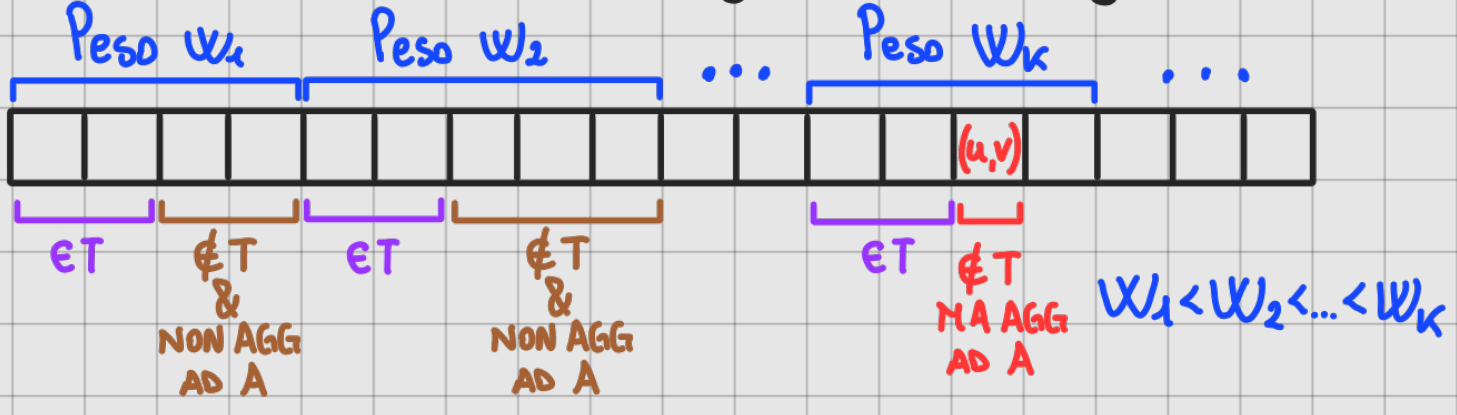
Esercizio 9. L'algoritmo di Kruskal può restituire alberi di connessione differenti per lo stesso grafo in input G. Dimostrate che, per ogni albero di connessione minimo T di G, c'è un modo di ordinare gli archi di G nell'algoritmo di Kruskal che permette all'algoritmo di restituire T.

Sia T un MST di G che vogliamo ottenere come output di Kruskal. Ordiniamo gli archi di G con i seguenti criteri:

1. Ordine NON decrescente di peso
2. A parità di peso, gli archi di T vengono prima degli archi NON in T.

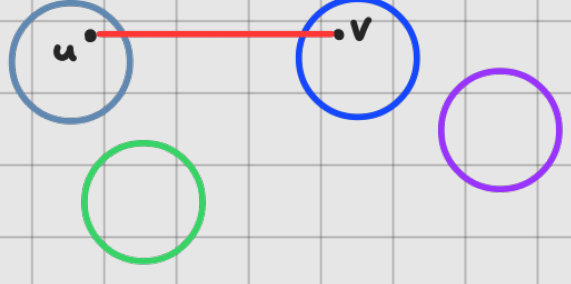
Supponiamo per assurdo che Kruskal con questo ordinamento aggiunga ad A un arco $(u,v) \notin T$. Sia (u,v) il primo arco con questa proprietà aggiunto ad A.

La situazione nell'ordinamento degli archi è la seguente:



Oss: Tutti gli archi in T successivi a (u,v) hanno $w(\cdot,\cdot) > w(u,v)$

Se (u,v) è aggiunto ad A, allora u & v si trovano in componenti distinte della foresta indotta da A in quel momento.



↳ $A = \{$ T-archi che precedono $(u,v)\}$

Se prendiamo A e aggiungiamo (u,v) più tutti i T-archi che vengono dopo (u,v) nell'ordinamento, otteniamo esattamente $T \cup \{(u,v)\}$: ha $\underbrace{n-1}_{T} + \underbrace{1}_{(u,v)} = n$ archi e quindi contiene esattamente un ciclo.

Il ciclo deve passare per qualche arco e che viene dopo (u,v) nell'ordinamento: se tutti gli archi del ciclo venissero prima, u e v sarebbero già stati connessi in A prima di aggiungere (u,v), e Kruskal non lo avrebbe mai accettato.

Tale arco $e \in T$ soddisfa $w(e) > w(u,v)$ per costruzione.

Rimuovendo e dal ciclo otteniamo

$$T' = T \cup \{(u,v)\} \setminus \{e\}$$

un albero di copertura con

$$w(T') = w(T) - w(e) + w(u,v) < w(T)$$

Contraddizione con la minimalità di T.